

Online Video Conferencing Application

Project Description:

MeetSpace is a full-stack, real-time online video meeting application designed to provide a seamless, browser-based communication platform. It is built using the MERN stack, utilizing React.js for the frontend, Node.js and Express.js for the backend, and MongoDB for secure user data management. To enable high-performance, low-latency peer-to-peer and multi-party video and audio streaming, the platform leverages the Agora RTC SDK.

Scenario:

Consider a scenario where a remote team of professionals or a group of students needs to collaborate in real time without relying on heavyweight commercial tools. A user visits MeetSpace, registers an account with their name and email, and logs in to receive a secure JWT token. From the Home dashboard, the user either creates a new meeting by entering a custom room name, or joins an existing session by entering a shared room code.

System Requirement:

1. Software Requirements

These are the essential platforms and tools required to manage the frontend logic, server-side routing, and real-time communication:

- **Operating System:** Windows 10/11, macOS, or Linux to support cross-platform development.
- **Environment & Runtime:**
 - **Node.js (v16 or above):** Powers the server-side logic and handles API routing.
 - **npm (v8 or above):** Essential for installing dependencies for React and other libraries.
- **Frontend Technologies:**
 - **React.js:** Used for building the dynamic and responsive user interface.
 - **Tailwind CSS:** Used for the application's modern look and layout.
- **Backend & Database:**
 - **Express.js:** A lightweight framework used to build RESTful APIs.
 - **MongoDB:** A NoSQL database used to store user records and resource data.

- **Communication & Security:**

- **Agora RTC SDK:** The primary tool for enabling real-time peer-to-peer and multi-party video and audio streaming.
- **JSON Web Token (JWT):** Used for secure, stateless user authentication and session management.

- **Development Tools:**

- **Visual Studio Code (VS Code):** The preferred code editor.
- **Postman:** Used for testing API endpoints during development.
- **Git & GitHub:** For version control and collaborative development.
- **Browser:** Latest versions of Google Chrome or Firefox for rendering and testing the UI.

2. Hardware Requirements

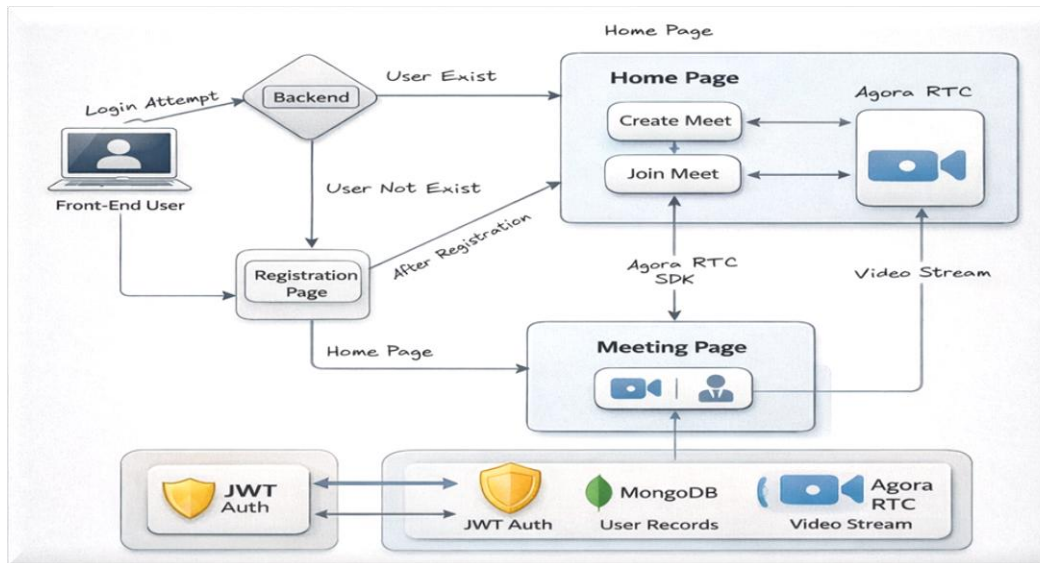
These specifications ensure the system can handle development servers, IDEs, and real-time streaming simultaneously:

- **Processor:** Intel Core i5 (8th Gen or above) or AMD Ryzen 5 (or better) to ensure fast compilation and multitasking.
- **RAM: Minimum 8 GB**, though **16 GB is highly recommended** to handle the load of development servers and browser testing.
- **Storage:** At least **1 GB of free space** for package installations, MongoDB setup, and project files.
- **Display:** A resolution of **1366x768 or higher** for optimal coding and UI visualization.
- **Network: Stable broadband internet** is strictly required for the Agora RTC real-time audio and video streaming to function without lag

Project Architecture:

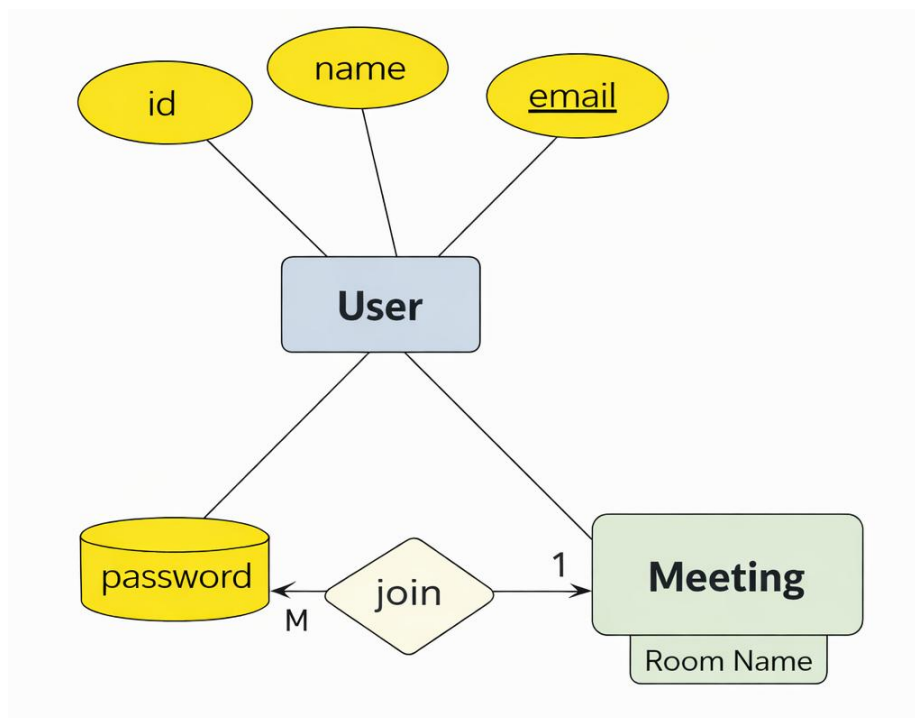
1. Technical Architecture:

- When a user accesses MeetSpace, the frontend sends a login request to the backend API.
- The backend checks whether the user exists in the MongoDB database.
- If the user is new, they are redirected to the Registration Page where their details are securely stored using bcrypt hashing.
- After successful authentication, a JWT token is generated and sent to the client for session management.



- On the Home Page, users can either Create Meet or Join Meet by entering a meeting ID.
- The frontend communicates with the backend APIs to validate and process the request.
- Once inside the Meeting Page, the Agora RTC SDK is initialized to establish real-time audio and video communication.
- Users can control microphone and camera settings dynamically.
- The JWT token is used to protect routes and ensure only authenticated users can access meeting features.
- MongoDB stores user credentials and related session data, ensuring persistence and scalability.

2. ER Diagram:



- The ER model consists of two primary entities: User and Meeting.
- The User entity contains attributes such as id, name, email, and password, where the password is stored as a hashed value for security.
- The Meeting entity is identified by a unique room name, which users enter to join a session.
- A relationship named "Join" connects User and Meeting.
- The relationship follows a Many-to-One (M:1) cardinality, meaning multiple users can join a single meeting room.
- This relationship is dynamically activated when users enter a meeting room name, after which real-time communication is handled using the Agora RTC service.

3.Features:

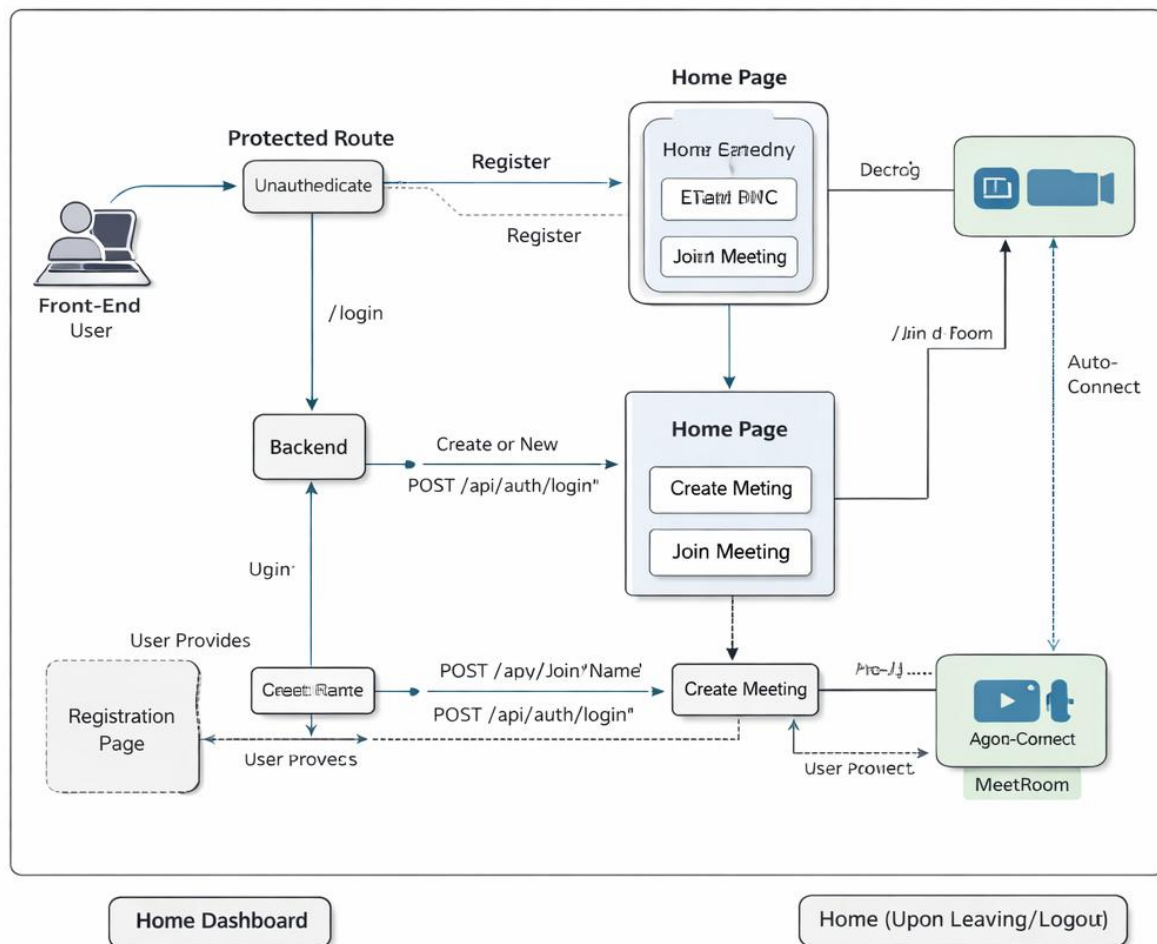
- **Secure Authentication:** JWT-based token system ensures that only registered users can access the dashboard.
- **Dynamic Room Creation:** Users can generate unique meeting rooms on the fly by simply typing a name.
- **High-Quality Media:** Leveraging Agora's Global Network for HD video and crystal-clear audio.
- **In-Call Controls:** Real-time toggling of local hardware (Camera/Mic) without interrupting the stream.
- **Participant Grid:** Responsive layout that automatically adjusts as new users join or leave.
- **Cross-Device Stability:** Built with Tailwind CSS to ensure the UI remains functional on mobile and desktop.

4. Roles and Responsibilities:

MeetSpace is designed with a streamlined permission model.

Role	Responsibilities
Guest	Can view the landing page, register a new account, or log in. Forbidden from viewing the dashboard or entering meetings.
Authenticated User	Can generate meeting links, join existing sessions, and manage their own media tracks (mute/camera). They are responsible for sharing the room Code with other participants.
Meeting Host	(Internal Logic) The user who creates the room; currently holds the same capabilities as a standard user but is tracked for logging purposes.

5. User Flow:

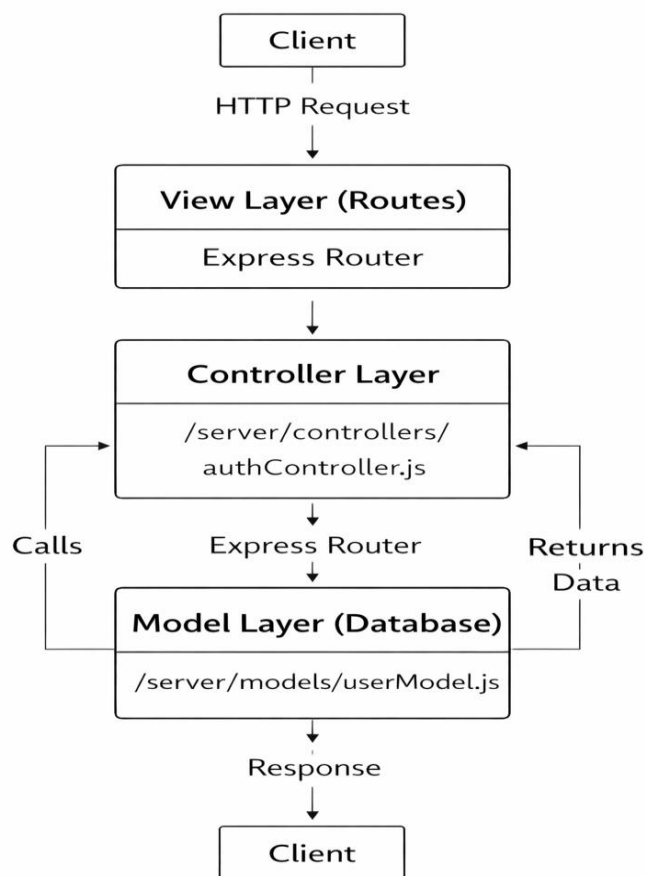


- **Initial Visit & Authentication Check:**
When a user accesses MeetSpace, Protected Routes verify authentication.
Unauthenticated users are redirected to the /login page.
- **Registration:**
New users register by submitting their name, email, and password via POST /api/auth/register.
- **Login:**
Users log in using POST /api/auth/login. Upon success, a JWT token and username are stored in local Storage, and the user is redirected to the Home page.
- **Home Dashboard:**
 - The user sees two options: Create Meeting or Join Meeting.
- **Entering a Meeting:**
Create: /meet:/room Name
Join: /meet:/ room Code
- **Connection Establishment:**
The Agora RTC client joins the channel and publishes audio/video streams.

- **Real-Time Interaction:**
Participants are auto-connected, and video streams appear dynamically.
- **In-Call Controls:**
Users can toggle microphone and camera during the session.
- **Leaving Meeting:**
Disconnects from Agora and returns to Home.
- **Logout:**
JWT token is removed, and the user is redirected to /login.

MVC Pattern:

MVC Pattern in MeetSpace



The MeetSpace backend follows the **MVC (Model–View–Controller)** architecture to organize the code into separate layers. This improves maintainability, scalability, and clarity of logic.

1. Model Layer (Data Layer)

This layer is responsible for all **data-related logic**, including the definition of data schemas and database operations.

- **Implementation:** In MeetSpace, the Model is implemented using **Mongoose** to define schemas for MongoDB.

- **Example:** The **User model** (found in `/server/models/userModel.js`) specifies the shape of a user document, including the user's name, email, and password (stored as a **bcrypt hash**). All MongoDB queries, such as `create`, `findOne`, and `find`, are executed through this model.

2. View Layer (Routing Layer)

In a backend REST API context, the **View** is implemented as the **routing layer**.

- **Implementation:** It defines the various **API endpoints** using **Express Router**.
- **Function:** This layer determines how the backend responds to different HTTP requests (such as **GET or POST**) and is responsible for invoking the appropriate controller functions.
- **Example:** MeetSpace defines endpoints like `POST /api/auth/register` and `POST /api/auth/login` in `/server/routes/authRoute.js`.

3. Controller Layer

The Controller is considered the "**brain**" of the application, acting as an intermediary between the View and the Model.

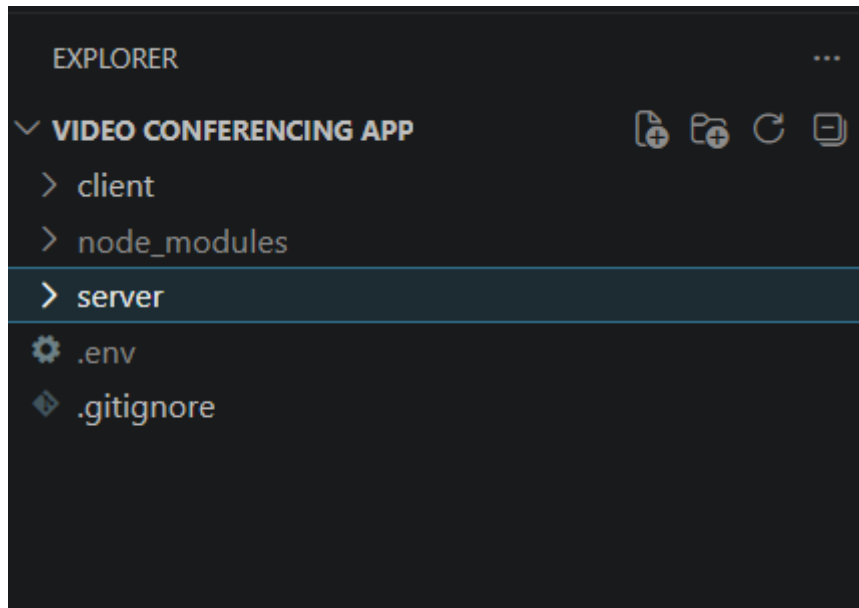
- **Implementation:** Located in `/server/controllers/authController.js`, it contains functions like `register` and `login`.
- **Function:** It receives incoming requests, processes input (including **validation or transformation**), calls the necessary methods from the model, and returns a response to the client.
- **Business Logic:** It handles all core logic, such as checking for **duplicate emails**, hashing passwords with **bcrypt**, and signing **JWT tokens** for session management.

Advantages of MVC in MeetSpace

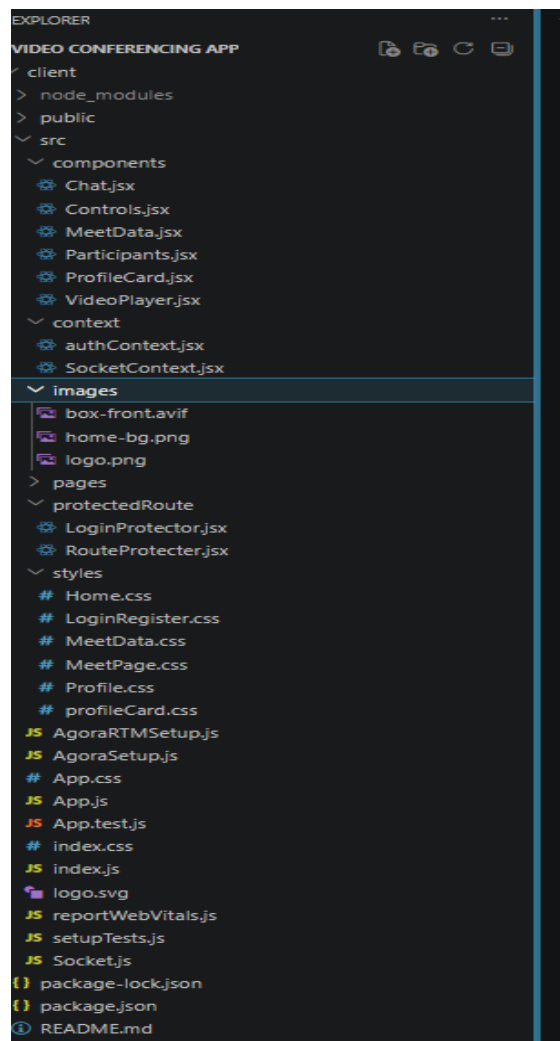
- **Separation of Concerns:** Each layer has a clearly defined responsibility, which improves code readability and maintainability.
- **Scalability:** New features can be added easily by creating new routes, controllers, and models without affecting existing layers.
- **Independent Testing:** Each layer can be tested independently, particularly the logic within models and controllers.
- **Reusability:** Logic defined in controllers and models can be reused across different parts of the application.

Project Setup and Configuration:

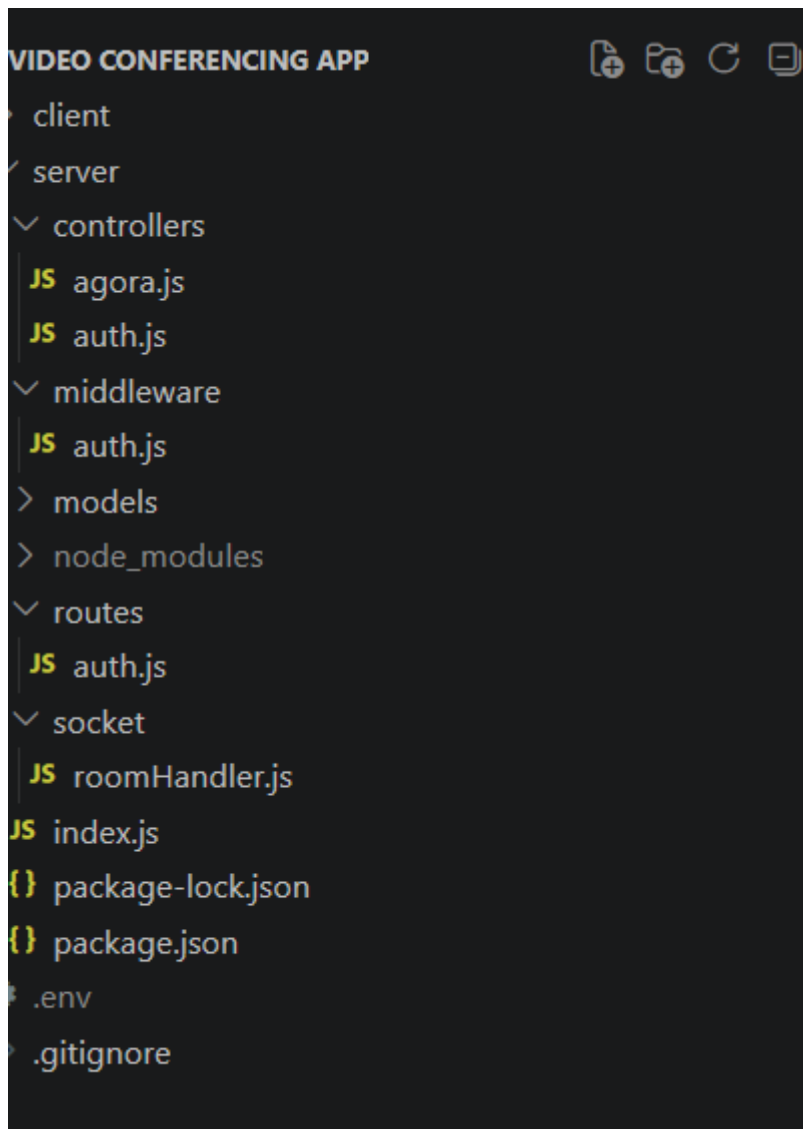
1. Folder Structure:



2. Client Folder Structure:



3. Server Folder Structure:



Client setup (installing react app)

- Open the Client folder in the terminal of VScode.

npm create vite@latest. -- --template react

- Select React framework from the given options.
- Select a *framework*:

| *React*

- Select JavaScript variant from the given options.

Select a variant:

JavaScript

- Now lets navigate to the client folder by giving the following

command.

cd client

- To install all the packages run the following command.

npm install

- To start the React server type the following command.

npm run dev

Server setup (npm init)

- Open Server folder in terminal of VScode.

npm init -y

- Create files:

server.js

- Create folders:

models

controllers

middleware

socket

routes

Backend Development:

https://drive.google.com/file/d/1ho5sUbwjx3Sw2Fditk_iA39yZKF_wMU9/view?usp=drive_link

Database Development:

https://drive.google.com/file/d/1ho5sUbwjx3Sw2Fditk_iA39yZKF_wMU9/view?usp=drive_link

Front-End Development:

https://drive.google.com/file/d/1ho5sUbwjx3Sw2Fditk_iA39yZKF_wMU9/view?usp=drive_link

Project Execution:

Step 1 - Download and Open the Project

- Clone or download the project folder from the repository.
- Open the project root folder in VS Code.
- You will see two sub-folders: client (React frontend) and server (Node.js backend).

Step 2 - Set Up the Frontend (React)

- Open a terminal and navigate into the client folder:

cd client

- Install all required frontend packages:

npm install

- Start the React development server:

npm run dev

- *The app should now be running at: <http://localhost:5173>*

Step 3 - Set Up the Backend (Express)

- Open a new terminal tab/window and navigate into the server folder:

cd..

cd server

- Install all backend dependencies:

npm install

Step 4 - Configure Environment Variables

- Inside the server folder, create a new file named `.env` (no file extension).
- Add the following environment variables:

MONGO_URI=mongodb://localhost:27017/video_conferencing_app_db

PORT=5000

JWT_SECRET="Any_JWT_key"

Step 5 - Start the Backend Server

- Inside the server folder, start the backend using :

node index.js

The server should start on: <http://localhost:5000>

Step 6 - Open the Application

Demo Video Link:

https://drive.google.com/file/d/1ho5sUbwjx3Sw2Fditk_iA39yZKF_wMU9/view?usp=drive_link

Code Link:

<https://github.com/Harshit-kumar-616/Video-Conferencing-App>